DNS and Route 53 for Domain Management

Practical on Route53

1.Create a Instance

1. Name ----------> webserver
2. Instance type -------> t3. micro
3. Ami ----------> ubuntu
4. Create a instance

2.Access The Instance

1. apt update
2. apt install apache2 -y
3. systemctl start apache2
4. cd /var/www/html/
5. rm -rf index.html
6. vim index.html
   <h1> Hello World </h1>
7. systemctl restart apache2

3.Check It on The Browser

<public_ip>

4.Go To Route 53 & Create A Hosted Zone

1. create a hosted zone
2. name ----------> mayurcbz.space
3. zone ----------> public hosted zone
4. create a hosted zone

5.Add Nameservers on The Hostinger

1. copy these 4 nameserver from the hosted zone
2. Hostinger -> Domain -> Domain portfolio -> Domain name -> DNS nameservers -> change nameservers -> delete all the nameservers -> add our copied nameservers -> save
3. Wait for up to 10 minutes or 24 hours to update the nameservers

6.Go to Route 53

1. Click on hosted zone
2. Create a record
3. Record name --------> (empty)
4. Record type ----------> A

5. Value ----------> <server public_ip)
6. Routing policy -------> simple routing policy
7. Create a record

ROUTING POLICY

1.Simple Routing Policy

1. This policy is used when you have only one resource, like a single server or website.
2. Route 53 simply returns the same IP address every time without any decision-making.
3. It does not support load balancing or failover, so it's very basic.
4. Best suited for small or static websites where high availability is not required.

2.Failover Routing Policy

1. This policy is designed to provide high availability by using a primary and a backup resource.
2. Route 53 continuously checks the health of the primary server using health checks.
3. If the primary fails, traffic is automatically redirected to the secondary (failover) server.
4. It ensures your application remains accessible even during failures.
5. Example
    a. 10.0.0.0 --------> primary server (active server)
    b. 20.0.0.0 --------> secondary server (passive server)

CLOUDBLITZ

3.Weighted Routing Policy

1. This policy allows you to split traffic between multiple resources based on assigned weights.
2. For example, you can send 80% of traffic to one server and 20% to another.
3. It is commonly used for testing new versions of applications (A/B testing).
4. Helps in gradual traffic shifting without affecting all users at once.
5. Example:
    a. 10.0.0.0 ----------> 50%
    b. 20.0.0.0 ----------> 60%
    c. 30.0.0.0 ----------> 20%

4.Latency Based Routing Policy

1. This policy routes users to the AWS region that provides the lowest network latency.

2. It improves the speed and performance of your application for global users.
3. Route 53 makes decisions based on real-time latency measurements.
4. Ideal for applications where fast response time is critical.

5.Geolocation Routing Policy

1. This policy routes traffic based on the geographic location of the user (country or continent).
2. It allows you to serve region-specific content, such as language or legal restrictions.
3. You can also restrict access from certain locations if needed.
4. Useful for businesses that target different audiences in different regions.
5. Example:
    a. 10.10.10.10 ------> India ----> Hindi
    b. 20.20.20.20 ------> US -------> Japanese
    c. 30.30.30.30 ------> Japan ---> English

6.Geoproximity Routing Policy

1. This policy routes traffic based on the distance between users and your resources.
2. You can adjust traffic flow using a bias value to increase or decrease traffic to a region.
3. It provides more control compared to geolocation routing.
4. Commonly used for balancing traffic across multiple geographic regions.

7.Multi-Value Answer

1. This policy returns multiple IP addresses instead of just one.
2. Route 53 only returns healthy resources, improving reliability.
3. It works like a simple load balancing mechanism.
4. Helps increase availability and fault tolerance of applications.
5. Example:
    a. 10.0.0.0
    b. 20.0.0.0
    c. 30.0.0.0
    d. 40.0.0.0
6. So I have added 4 servers in the backend so users get the response randomly from any of the server but only from the healthy server.

8.IP-Based Routing Policy

1. This policy routes traffic based on the client's IP address range (CIDR blocks).
2. You can define custom rules for specific networks or users.
3. Useful for security, compliance, or internal routing needs.

4. Helps deliver personalized or restricted access based on IP ranges.

ROUTE-53 WITH LOAD BALANCER & WAF & CREATING SSL CERTIFICATE

1.Create A Instance

1.nam --------> app1
2.machine ---> amazon-linux
3.az--------1a
4.sg--------http,https,ssh
5.useradata-

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo systemctl enable httpd.service
sudo echo "SERVER-1" > /var/www/html/index.html
sudo systemctl restart httpd.service
```

6.create a instance

2.Create A Instance

1.name----app2
2.machine---amazon-linux
3.az--------1b
4.sg--------http,https,ssh
5.useradata-

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo systemctl enable httpd.service
sudo echo "SERVER-2" > /var/www/html/index.html
sudo systemctl restart httpd.service
```

6.create a instance

3.Create A Target Group

1. Type------> instance
2. name-----> tg
3. path-----/
4. register---> 1.app1

CLOUDBLITZ

2.app2

5. create a target group

4.Create A Load Balancer

1. name----ALB
2. vpc-------default
3. az-----1a,1b,1c
4. sg-----our instance sg
5. forward to ----> tg
6. create a load balancer

5.Go To Route-53

1.create a hosted zone

2.copy the all name-server and paste it on the Hostinger account

6.Go To Hostinger

1. Go to Domain → Nameservers
2. Replace with Route 53 NS records

7.Create Alias Record In Route53

1.create a record

1. name----------> (empty)
2. route traffic---> alias
3. endpoint--------> (region-name ---> load balancer dns name)
4. policy----------> simple routing
5. create a record

CLOUDBLITZ

8.Check It On The Web

http://mayurcbz.space

9.Create ACM (SSL) Certificate

1.request certificate

2.domain------> mayurcbz.space

3.Validation → DNS

3.request a certificate

4.view certificate

5.create a record in route-53

6.check status -----> Issued

10. Add HTTPS to Load Balancer

1. add listener
2. add port ---------> https-443
3. Forward to -----> tg
4. add acm ----> (certificate-mayurcbz.space)

11. Redirect HTTP → HTTPS

1.edit already created listener -> 80
2.redirect to URL ------> https --> (443)
3.save

12.Check It On The Web

{ https://mayurcbz.space/ }

13.Creating A WAF For Load Balancer

1.Create A IP Set

1. IP set name ----------> my-ip-set
2. scope ----------> regional
3. ip version ----------> IPv4
4. ip address ----------> (my system ip address/32 – find my ip address)
5. save

2.Create A WEB ACL

1. app category ---------> others
2. app focus ----------> web
3. add resource ----> add regional resource ----> select ALB ----> add
4. click on recommended rules for u
5. name ----------> my-web-acl
6. create a web acl

3.Create A Rule

1. click on my-web-acl
2. click on manage rule ----------> add rule
3. custom rule ---> Next
4. ip based rule ----> Next
5. Action ----------> Block
6. Rule name -------> my-rule
7. Statement -------> Use existing IP set -------> my-ip-set
8. Add rule

Note:

1. At these time u have been block u get the 403 forbidden error

CLOUDBLITZ

4.Change the rule

1. click on my-web-acl
2. click on manage rule -------> my-rule
3. Action ------> captcha

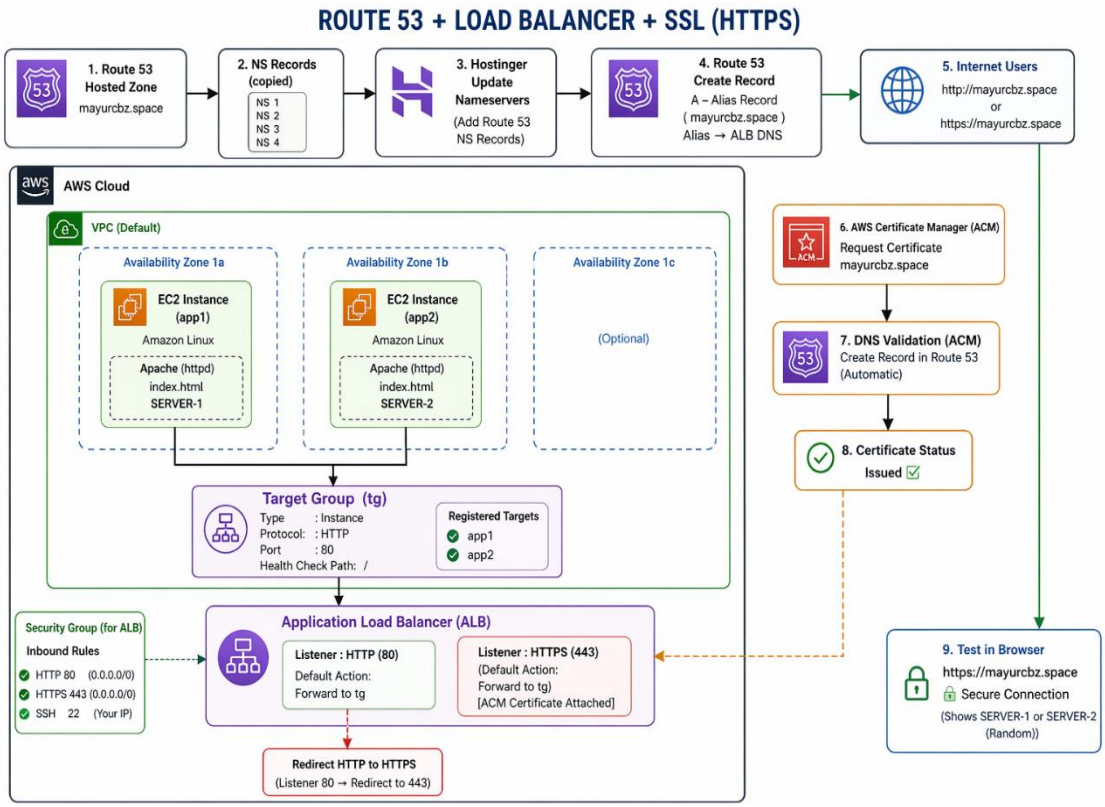
4. If a request -----> does not match the statement (NOT)

5.inspect ----------> originates from an ip address in -------> my-ip-set

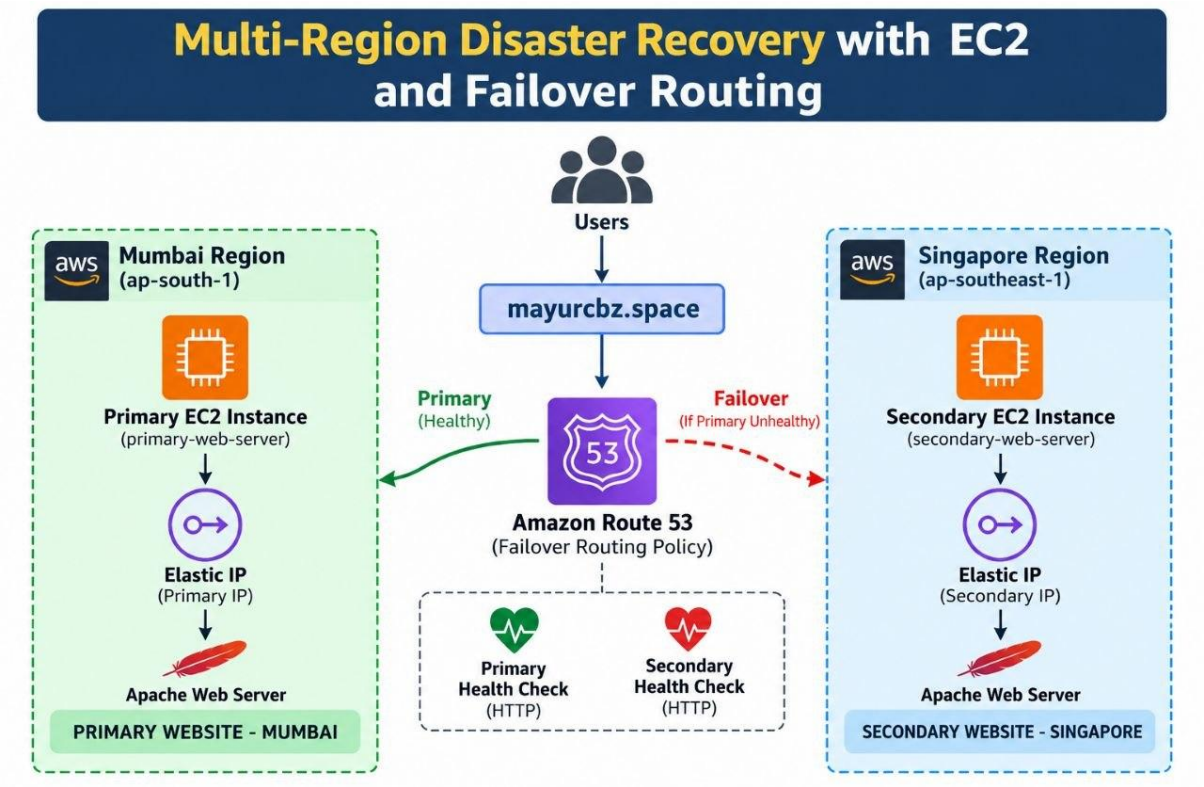
6.save rule

Note:

1. At this time all the users get the captcha first before going into the website but only your ip directly go the website



MULTI-REGION DISASTER RECOVERY WITH EC2 AND FAILOVER ROUTING



LAUNCH PRIMARY EC2 INSTANCE IN MUMBAI

1.create a instance

1. name - primary-web-server
2. ami - ubuntu
3. instance_type - t3.micro
4. key_pair - mayur
5. sg - ssh/http
6. user_data

   ```
   #!/bin/bash
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>PRIMARY WEBSITE - MUMBAI</h1>' | sudo tee
   /var/www/html/index.html
   sudo systemctl enable --now apache2
   ```
7. launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address

3. associate IP with primary-web-server

LAUNCH SECONDARY EC2 INSTANCE IN SINGAPORE

1.create a instance

1. name - secondary-web-server
2. ami - ubuntu
3. instance_type - t3.micro
4. key_pair - mayur
5. sg - ssh/http
6. user_data
   #!/bin/bash
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>SECONDARY WEBSITE - SINGAPORE</h1>' | sudo tee /var/www/html/index.html
   sudo systemctl enable --now apache2
7. launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with secondary-web-server

ROUTE53

CREATE THE PRIMARY AND SECONDARY HEALTH CHECK

1.create a health check

1. name - primary-mumbai-health
2. monitor - endpoint
3. endpoint - IP address
4. protocol - HTTP
5. IP address - (primary-elastic-ip-address/)
6. create health check

2.create a health check

1. name - secondary-singapore-health
2. monitor - endpoint
3. endpoint - IP address
4. protocol - HTTP
5. IP address - (secondary-elastic-ip-address/)
6. create health check

ROUTE53

CREATE THE PRIMARY & SECONDARY FAILOVER RECORD

1.click on record

1. record name - leave blank
2. record type - A
3. value - (primary-elastic-ip-address)
4. routing policy - failover
5. failover record type - primary
6. Health check - primary-mumbai-health
7. Record ID - (paste primay-mumbai-health-id)
8. create record

1.click on record

1. record name - leave blank
2. record type - A
3. value - (secondary-elastic-ip-address)
4. routing policy - failover
5. failover record type - secondary
6. Health check - secondary-singapore-health
7. Record ID - (paste secondary-singapore-health-id)
8. create record

TEST

1. http://mayurcbz.space
   (you get only the primary Mumbai server)

CLOUDBLITZ

SIMULATE PRIMARY FAILURE

1. Connect to the Mumbai primary EC2 and run:
2. sudo systemctl stop apache2

TEST

1. http://mayurcbz.space
   (you get the secondary server)

RESTORE THE PRIMARY AND TEST FAILBACK

On the Mumbai primary EC2 instance, run:

1. sudo systemctl start apache2
2. sudo systemctl status apache2

TEST

1. http://mayurcbz.space

2. (you get back to Primary server)

HOW DO WE REDUCE DOWNTIME

Improve Route 53 failover

- Use a 60-second TTL.
- Use 10-second fast health checks if appropriate for your budget.
- Reduce the failure threshold cautiously.
- Keep the Singapore server running and ready.

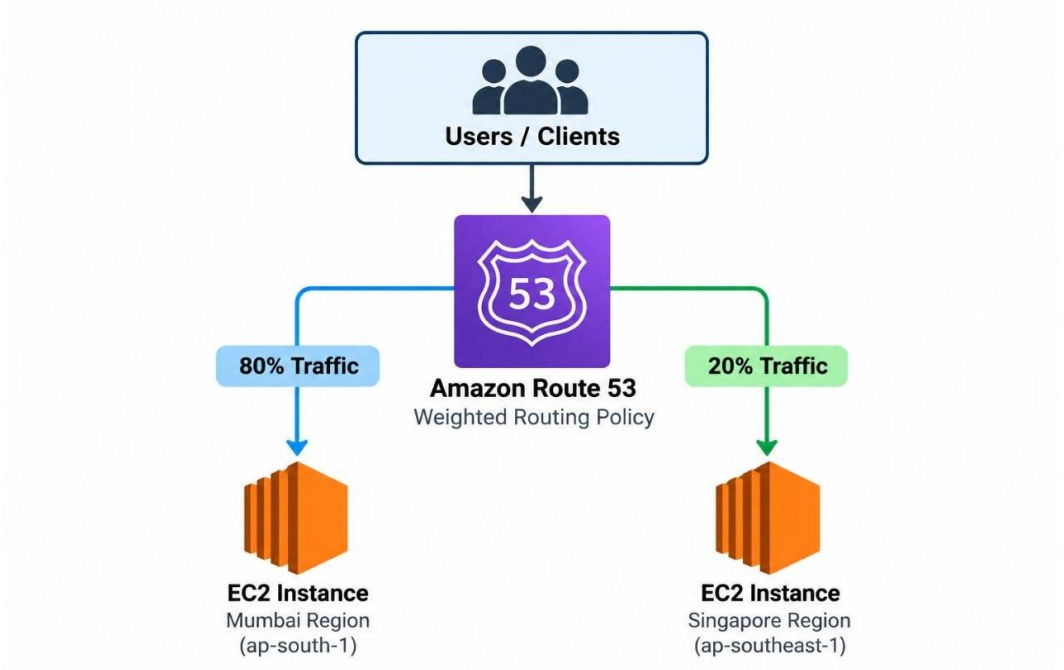
AWS Global Accelerator

- Give users two static anycast IP addresses.
- Register endpoints in Mumbai and Singapore.
- Configure health checks and endpoint groups.
- Send new connections to healthy endpoints without relying on DNS TTL changes for each failover.

Active-active multi-Region architecture

- Run the application in both Regions.
- Send traffic to both healthy Regions.
- Design the database, sessions, and application state for cross-Region resilience.
- Add retries, timeouts, and graceful error handling.

WEIGHTED ROUTING POLICY FOR MULTI-REGION TRAFFIC DISTRIBUTION



LAUNCH EC2 INSTANCE IN MUMBAI

1. create a instance
    1. name - primary-web-server
    2. ami - ubuntu
    3. instance_type - t3.micro
    4. key_pair - mayur
    5. sg - ssh/http
    6. user_data
       sudo apt update
       sudo apt install apache2 -y
       echo '<h1>PRIMARY WEBSITE - MUMBAI</h1>' | sudo tee /var/www/html/index.html
       sudo systemctl enable --now apache2
    7. Launch ec2 instance

Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with primary-web-server

LAUNCH EC2 INSTANCE IN SINGAPORE

1. create a instance
    2. name - secondary-web-server
    3. ami - ubuntu
    4. instance_type - t3.micro
    5. key_pair - mayur
    6. sg - ssh/http
    7. user_data
       sudo apt update
       sudo apt install apache2 -y
       echo '<h1>SECONDARY WEBSITE - SINGAPORE</h1>' | sudo tee /var/www/html/index.html
       sudo systemctl enable --now apache2
    8. launch ec2 instance

Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address

CLOUDBLITZ

3. associate IP with secondary-web-server

ROUTE53
CREATE THE MUMBAI WEIGHTED RECORD

click on record

1. record name - weighted
2. record type - A
3. value - (mumbai-elastic-ip-address)
4. routing policy - Weighted
5. Weight - 80
6. Record ID - mumbai-weight-80
7. create record

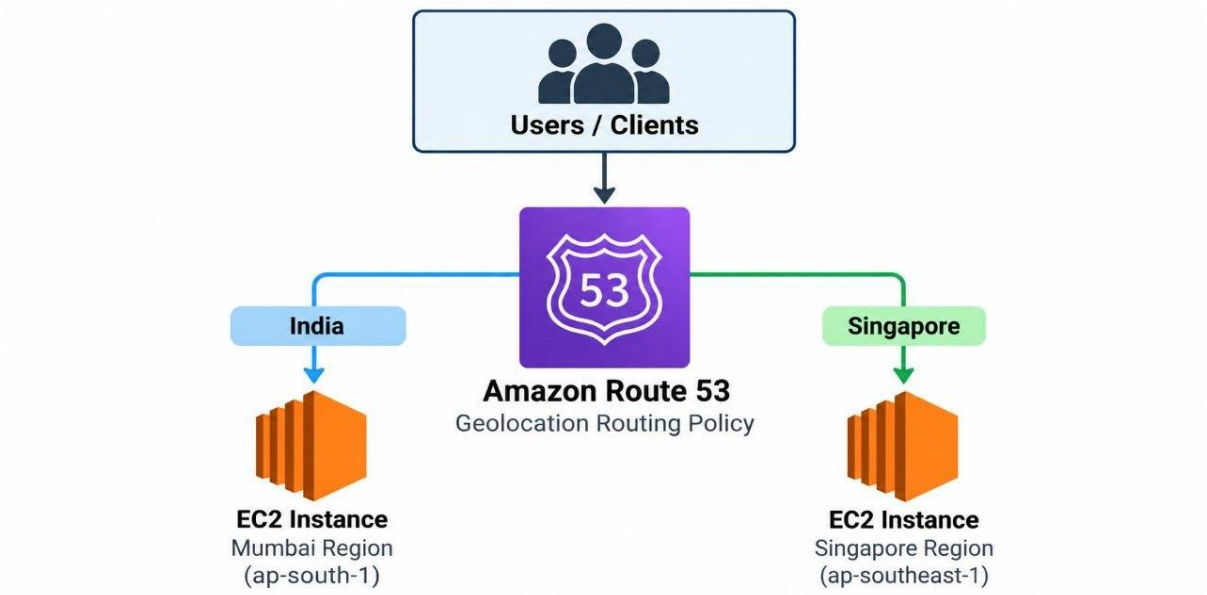
CREATE THE SINGAPORE WEIGHTED RECORD

1. click on record
    1. record name - weighted
    2. record type - A
    3. value - (Singapore-elastic-ip-address)
    4. routing policy - Weighted
    5. Weight - 20
    6. Record ID - Singapore-weight-20
    7. create record

TEST
http://weighted.mayurcbz.space

Identify the page you see:
- PRIMARY WEBSITE - MUMBAI
- SECONDARY WEBSITE - SINGAPORE

GEOLOCATION ROUTING FOR MULTI-REGION EC2 SERVERS



LAUNCH EC2 INSTANCE IN MUMBAI

1. create a instance
    1. name - primary-web-server
    2. ami - ubuntu
    3. instance_type - t3.micro
    4. key_pair - mayur
    5. sg - ssh/http
    6. user_data
       sudo apt update
       sudo apt install apache2 -y
       echo '<h1>PRIMARY WEBSITE - MUMBAI</h1>' | sudo tee /var/www/html/index.html
       sudo systemctl enable --now apache2
    7. launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with primary-web-server

LAUNCH EC2 INSTANCE IN SINGAPORE

create a instance

CLOUDBLITZ

1. name - secondary-web-server
2. ami - ubuntu
3. Instance_type - t3.micro
4. key_pair - mayur
5. sg - ssh/http
6. user_data
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>SECONDARY WEBSITE - SINGAPORE</h1>' | sudo tee /var/www/html/index.html
   sudo systemctl enable --now apache2
7. launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with secondary-web-server

ROUTE53

CREATE THE INDIA RECORD

click on record

1. record name - geo
2. record type - A
3. value - (mumbai-elastic-ip-address)
4. routing policy - Geolocation
5. TTL - 60
6. location - India
7. Record ID - india-mumbai
8. create record

CREATE THE SINGAPORE RECORD

click on record

1. record name - geo
2. record type - A
3. value - (singapore-elastic-ip-address)
4. routing policy - Geolocation
5. TTL - 60

CLOUDBLITZ

6. location - singapore
7. Record ID - Singapore
8. create record

CREATE THE DEFAULT RECORD

click on record

1. record name - geo
2. record type - A
3. value - (mumbai-elastic-ip-address)
4. routing policy - Geolocation
5. TTL - 60
6. location - singapore
7. Record ID - geo-default
8. create record

TEST

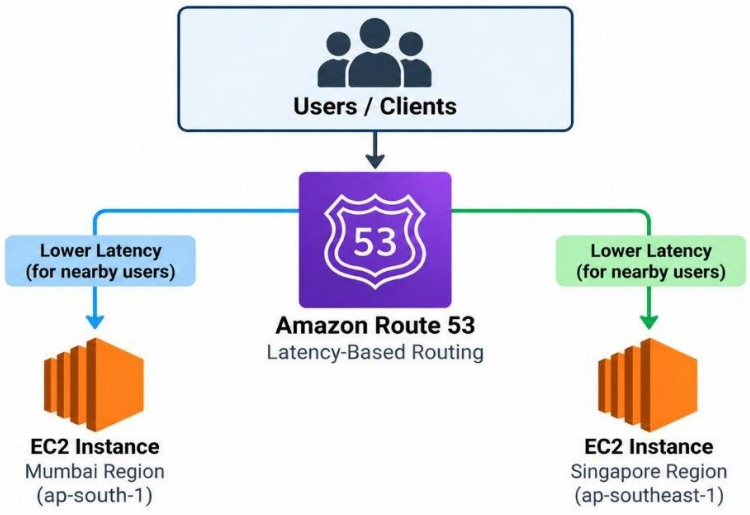
http://geo.mayurcbz.space

Understand the routing

1. India location - Mumbai
2. Singapore location - Singapore
3. other location – (default record – Mumbai)

CLOUDBLITZ

LATENCY-BASED ROUTING FOR MULTI-REGION EC2 SERVERS



LAUNCH EC2 INSTANCE IN MUMBAI

create a instance

1. name - primary-web-server
2. ami - ubuntu
3. instance_type - t3.micro
4. key_pair - mayur
5. sg - ssh/http
6. user_data
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>PRIMARY WEBSITE - MUMBAI</h1>' | sudo tee /var/www/html/index.html
   sudo systemctl enable --now apache2
7. Launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with primary-web-server

LAUNCH EC2 INSTANCE IN SINGAPORE

CLOUDBLITZ

create a instance

1. name - secondary-web-server
2. ami - ubuntu
3. instance_type - t3.micro
4. key_pair - mayur
5. sg - ssh/http
6. user_data
   sudo apt update
   sudo apt install apache2 -y
   echo '<h1>SECONDARY WEBSITE - SINGAPORE</h1>' | sudo tee /var/www/html/index.html
   sudo systemctl enable --now apache2
7. Launch ec2 instance

2.Allocate Elastic IP Address

1. click on elastic ip address
2. allocate elastic ip address
3. associate IP with secondary-web-server

CREATE THE MUMBAI LATENCY RECORD

click on record

1. record name - latency
2. record type - A
3. value - (mumbai-elastic-ip-address)
4. routing policy - latency
5. TTL - 60
6. Region - mumbai
7. Record ID - mumbai-latency
8. create record

CREATE THE SINGAPORE LATENCY RECORD

click on record

1. record name - latency
2. record type - A
3. value - (singapore-elastic-ip-address)
4. routing policy - latency
5. TTL - 60
6. Region - singapore
7. Record ID - singapore-latency
8. create record

CLOUDBLITZ

TEST

http://latency.mayurcbz.space
